Домашнее задание. Приложение: YouTube

1. Какие протоколы использует YouTube

Если смотреть по транспорту:

- **QUIC поверх UDP** — это основное сейчас. Он лежит в основе HTTP/3. В лекции как раз было, что YouTube передаёт контент через QUIC и получает ускорение загрузки на 20–30% и больше.
- **TCP** — остаётся как запасной вариант. Если QUIC не работает, YouTube откатывается на HTTP/2 или HTTP/1.1 поверх TCP. Ну и для чата, API, загрузки страниц тоже может использоваться.
- **UDP** — сам по себе тоже есть: DNS, телеметрия, иногда live-стримы через SRT/WebRTC. Но главное — это то, что QUIC работает поверх UDP.
- **RTMP/RTMPS, SRT, WebRTC** — это скорее для стримеров, когда они льют свой поток на YouTube. RTMP обычно через TCP, SRT и WebRTC — через UDP.

По прикладному уровню:

- **HTTP/3** — главный протокол для видео, картинок, API.
- **HTTP/2 и HTTP/1.1** — если HTTP/3 не завезли.
- **HTTPS/TLS 1.3** — шифрование. В QUIC оно встроено.
- **DASH / HLS** — адаптивная потоковая передача. Видео режется на кусочки, плеер выбирает качество по скорости.
- **WebSocket** — для чата и уведомлений.
- **gRPC/Protobuf** — может быть внутри инфраструктуры, но для юзера не видно.

2. Почему выбран такой стек и какие есть альтернативы

YouTube — это огромная платформа, поэтому ему нужно:

- много пропускной способности;
- низкая задержка;
- устойчивость к потерям;
- масштабируемость через CDN;
- безопасность;
- адаптивность под скорость пользователя.

Поэтому и используется **HTTP/3 поверх QUIC/UDP** с откатом на **TCP/HTTP/2**. QUIC даёт:

- быстрое соединение: 1-RTT, а если повторно — то 0-RTT;

- нет Head-of-Line Blocking, как в TCP;
- мультиплексирование потоков в одном соединении;
- встроенное шифрование TLS 1.3;
- соединение не рвётся при переходе с Wi-Fi на мобильную сеть;
- гибкое управление перегрузкой.

Для видео это важно: если пакет потерялся, QUIC не тормозит остальные потоки. Поэтому видео быстрее стартует и меньше буферизуется.

DASH/HLS нужны, чтобы подстраивать качество. Плеер сам решает, какой сегмент качать. Так можно раздавать видео через CDN миллионам людей.

TCP fallback нужен для совместимости — не везде есть QUIC/UDP.

Какие альтернативы и почему не подходят

- **Чистый TCP + HTTP/2** — надёжно, но есть Head-of-Line Blocking, больше задержка, хуже при потерях. Для видео это медленный старт и подвисания.
- **Чистый UDP без надёжности** — быстро, но нет гарантии доставки и порядка. Для массового видео не годится, придётся самому делать надёжность, а это уже QUIC.
- **WebRTC для всего видео** — хорош для звонков, но плохо масштабируется для VOD через CDN.
- **RTMP** — только для приёма потока от стримера, не для браузера.
- **MQTT/AMQP/Kafka** — это для сообщений и очередей, а не для видео.

3. Что даёт переход на QUIC/HTTP/3 и какие есть ограничения

YouTube уже использует QUIC/HTTP/3, но если представить переход, то плюсы такие:

- видео быстрее запускается за счёт 1-RTT/0-RTT;
- меньше задержка при переключении качества и в live-чате;
- меньше буферизации, потому что нет Head-of-Line Blocking;
- лучше работает в мобильных сетях — соединение не рвётся при смене Wi-Fi на LTE/5G;
- встроенная безопасность — TLS 1.3 шифрует всё;
- мультиплексирование — параллельные запросы не блокируют друг друга;
- управление перегрузкой можно менять без переделки сетевого железа.

Но есть и минусы:

- **UDP могут блокировать** корпоративные сети, провайдеры, firewall. Поэтому нужен fallback на TCP.
- **Больше нагрузка на CPU** из-за шифрования и обработки QUIC в user space.
- **Сложнее отлаживать** и мониторить, чем TCP.

- **Не все клиенты поддерживают QUIC** — старые устройства и браузеры требуют HTTP/2 или HTTP/1.1.
- **CDN и балансировщики должны поддерживать QUIC/HTTP/3**.
- **0-RTT имеет риски повторного воспроизведения**, надо аккуратно.
- **Шифрование QUIC мешает middlebox-оптимизациям** в сети.

4. Пример, где UDP лучше TCP

Лучший пример — **онлайн-игры** (CS2, Dota 2, Fortnite) или **VoIP** (Discord, Zoom).

Почему UDP:

- В играх и голосе важна **минимальная задержка**, а не доставка каждого пакета.
- Если пакет с позицией потерялся, старая информация уже не нужна — ждать её повторно бессмысленно.
- TCP при потере пакета останавливает всё остальное из-за Head-of-Line Blocking. В игре это были бы лаги и рассинхрон.
- UDP позволяет самому решать, что делать с потерями: важное передать надёжно, обычные обновления — без повторов.

Если нужна надёжность, её делают поверх UDP или берут QUIC. Но для чистой реальной передачи голоса, видео и игрового состояния UDP часто лучше TCP.